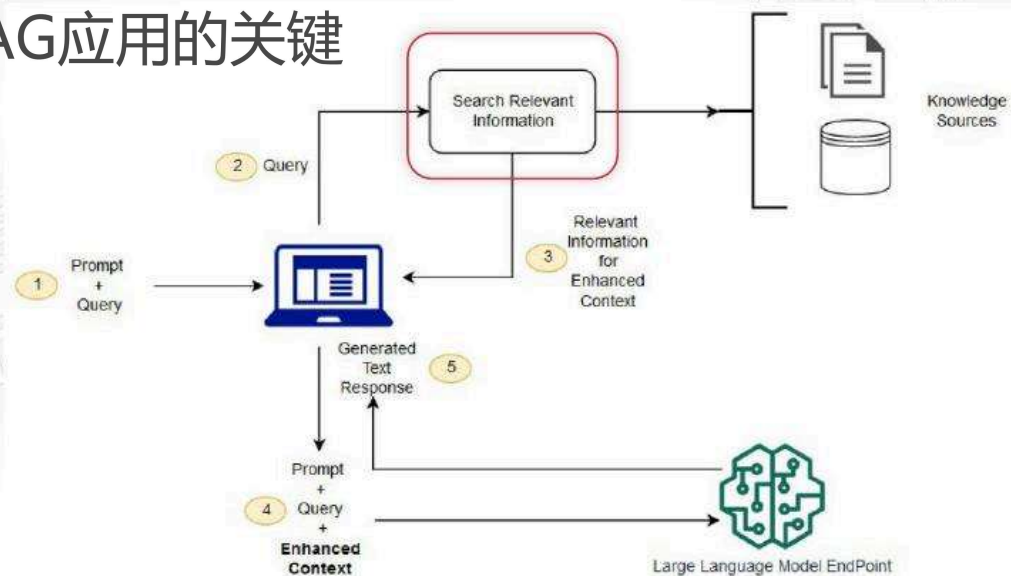


提升RAG的90分的检索增强技能

本章简介

- ◆ 检索（retrieval）关系到能否获得和用户问题相关的上下文信息，是RAG系统的重要组成，如何增强检索的性能是开发RAG应用的关键



本章简介

◆ 剖析RAG进化之路：探索优化点

◆ 实战



◆ 检索的两大形态：稀疏 vs 稠密

◆ 总结和展望：关于企业里需要良好的代码规范和代码管理

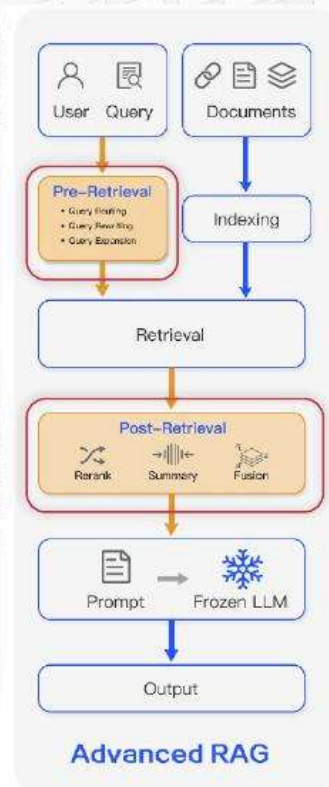
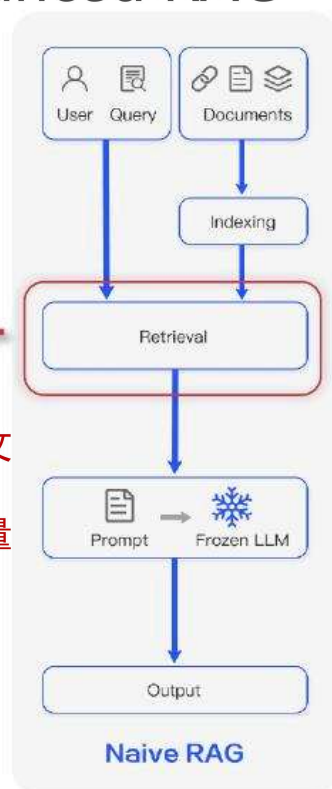
◆ 检索增强技术

剖析RAG进化之路：探索优化点

◆ naive vs advanced RAG

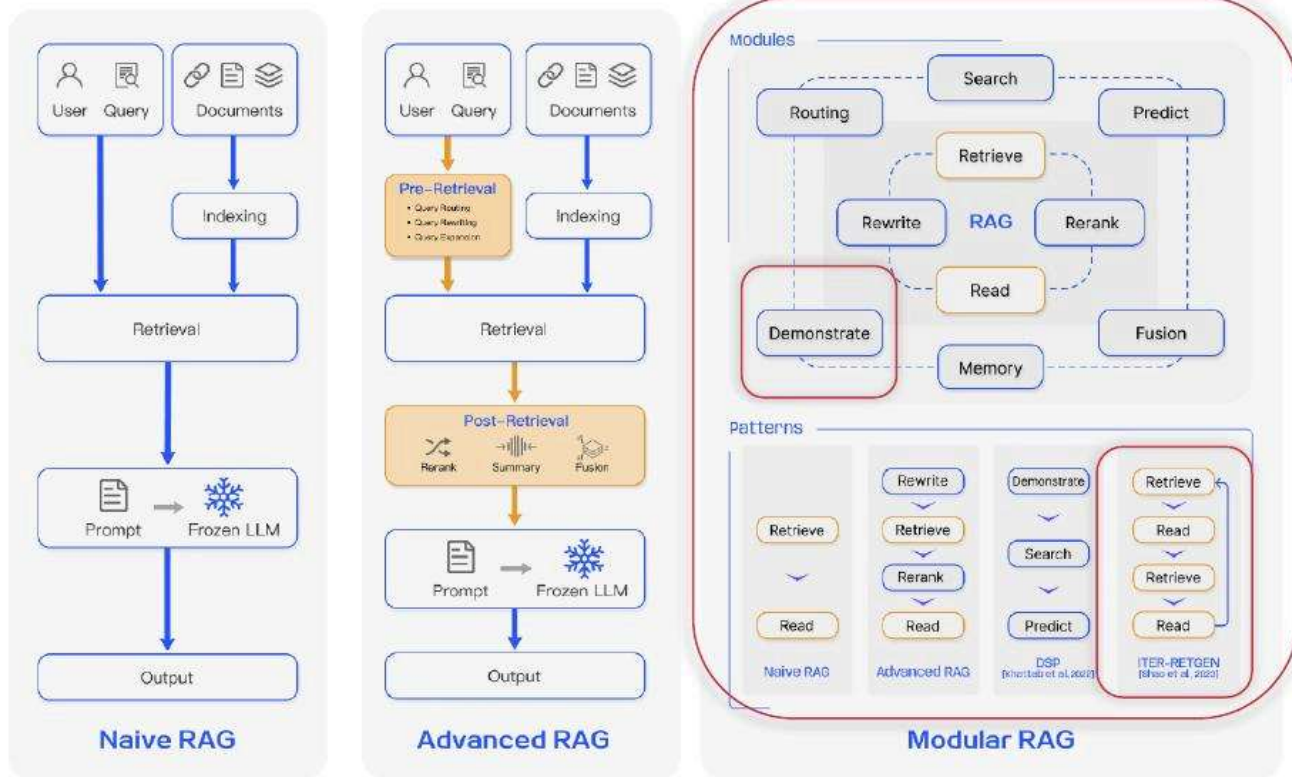
检索质量低

用户问题多样，无法匹配高质量上下文
不好的，有噪声的上下文会影响答案质量
...



剖析RAG进化之路：探索优化点

◆ Modular RAG



检索的两大形态：稀疏 vs 稠密

◆ 稀疏检索 (sparse retriever)

稀疏检索



关键词匹配的拓展

根据词频等信息，依赖于统计学方法，来评估文档与查询之间的相关性

常用算法：TF-IDF，BM25

检索的两大形态：稀疏 vs 稠密

◆ 稀疏检索 (sparse retriever)

TF-IDF

TF-IDF与一个词在文档中的出现次数成正比，与该词在整个语言中的出现次数成反比

特点：计算简单，只考虑词频，不关注顺序和上下文，词频来衡量文章中的一个词的重要性不够全面

度量词语在文档中的重要性

$$\text{词频(TF)} = \frac{\text{某个词在文章中的出现次数}}{\text{文章的总词数}}$$

$$\text{逆文档频率(IDF)} = \log\left(\frac{\text{语料库的文档总数}}{\text{DF 包含该词的文档数} + 1}\right)$$

反映了词语的普遍重要性。如包含词语的文档数很少，表示词语具有很好的类别区分能力，对于文本的重要性较高

权重，越常用的词越低

$$\text{TF-IDF} = \text{词频(TF)} \times \text{逆文档频率 (IDF)}$$

检索的两大形态：稀疏 vs 稠密

◆ 稀疏检索 (sparse retriever)

BM25



$$\sum_{t \in Q} \frac{\text{tf}_{t,d} \cdot (k+1)}{\text{tf}_{t,d} + k \cdot \left(1 - b + b \cdot \frac{l_d}{\text{mean}(l_d)}\right)} \cdot \ln \left(1 + \frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5}\right)$$

Best Match 25, TF-IDF的变种

TF

IDF

k 和 b 是参数，通常设置 $k \in [1.2, 2]$ 和 $b = 0.75$

检索的两大形态：稀疏 vs 稠密

◆ 稀疏检索 (sparse retriever)

考虑词频，不关注顺序和上下文

稀疏检索缺点 ----->

- 男朋友 / 送 / 的 / 礼物

- 送 / 男朋友 / 的 / 礼物

- 白 / 衬衫 / 灰 / 裤子

- 灰 / 衬衫 / 白 / 裤子

检索的两大形态：稀疏 vs 稠密

◆ 稠密检索 (dense retriever)

稠密检索



语义检索：将文本转换为embedding向量

基于BERT架构：GTE / BGE

信息压缩：将语义信息压缩到固定长度数组里，存在信息丢失的风险

检索增强技术

◆ 用户问题多样

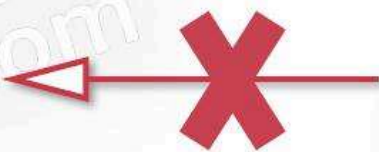
用户问题



知识库



上下文信息



1.问题存在表达不清、语义模糊, 缺少关键信息

2.问题不准确, 缺少上下文

3.问题本身具有一定歧义

4.包含多个问题, 问题太细节, ...



检索上下文精度差

检索增强技术

◆ 提高检索精度：查询增强(query rewriting/expansion)

查询增强

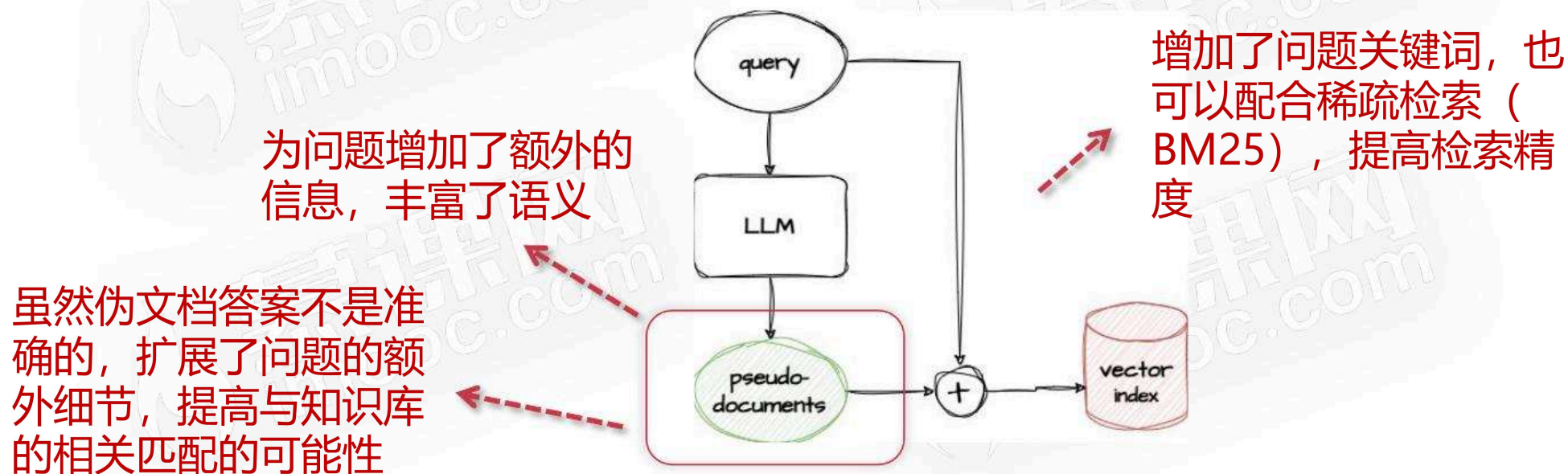
通过对用户查询进行改造和扩展，使其更加清晰、具体，从而提高检索准确性

使查询与文档语义一致

Pre-Retrieval: 检索前增强

检索增强技术

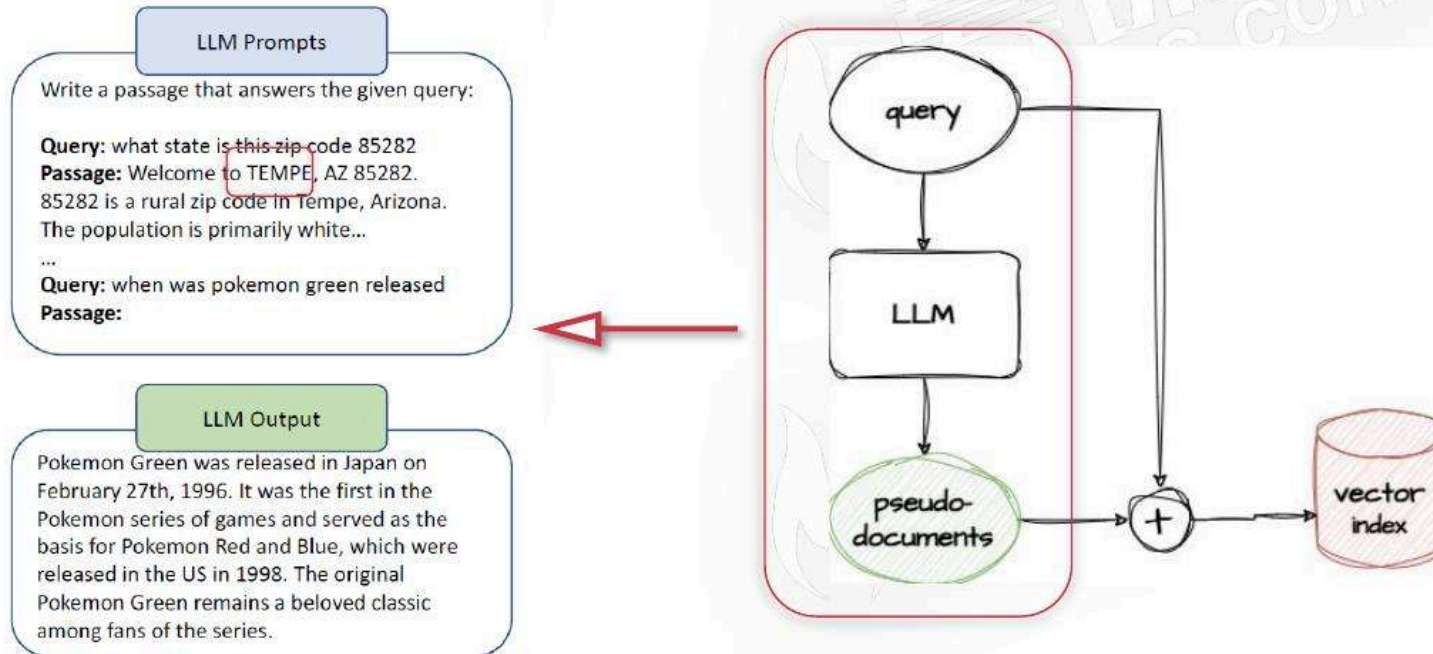
◆ 查询增强：Query2Doc



Query2doc: Query Expansion with Large Language Models

检索增强技术

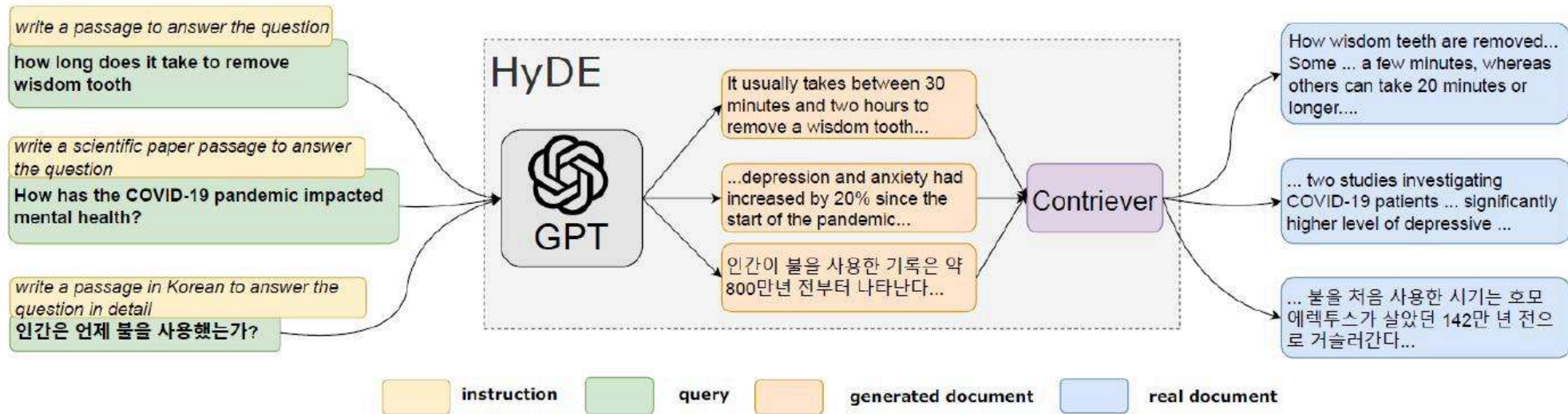
◆ 查询增强：Query2Doc



Query2doc: Query Expansion with Large Language Models

检索增强技术

◆ 查询增强: HyDE (假设性文档向量)



Hypothetical Document Embeddings (HyDE)

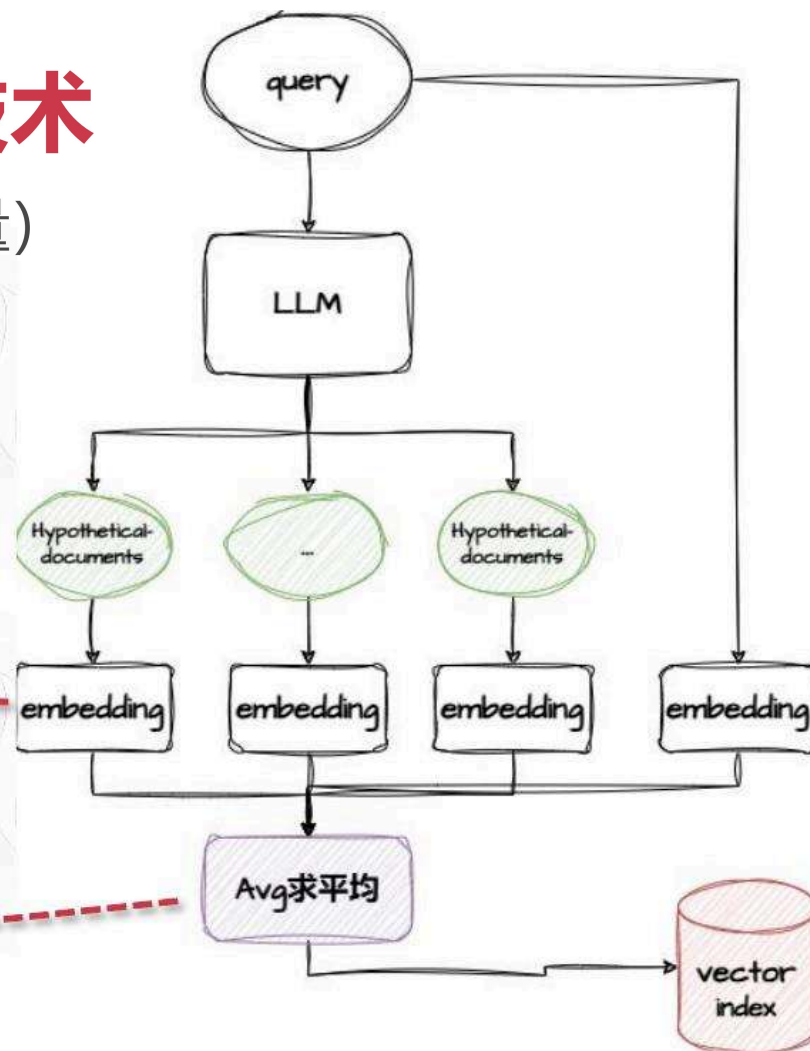
检索增强技术

◆ 查询增强：HyDE（假设性文档向量）

为问题增加了额外的信息，丰富了语义

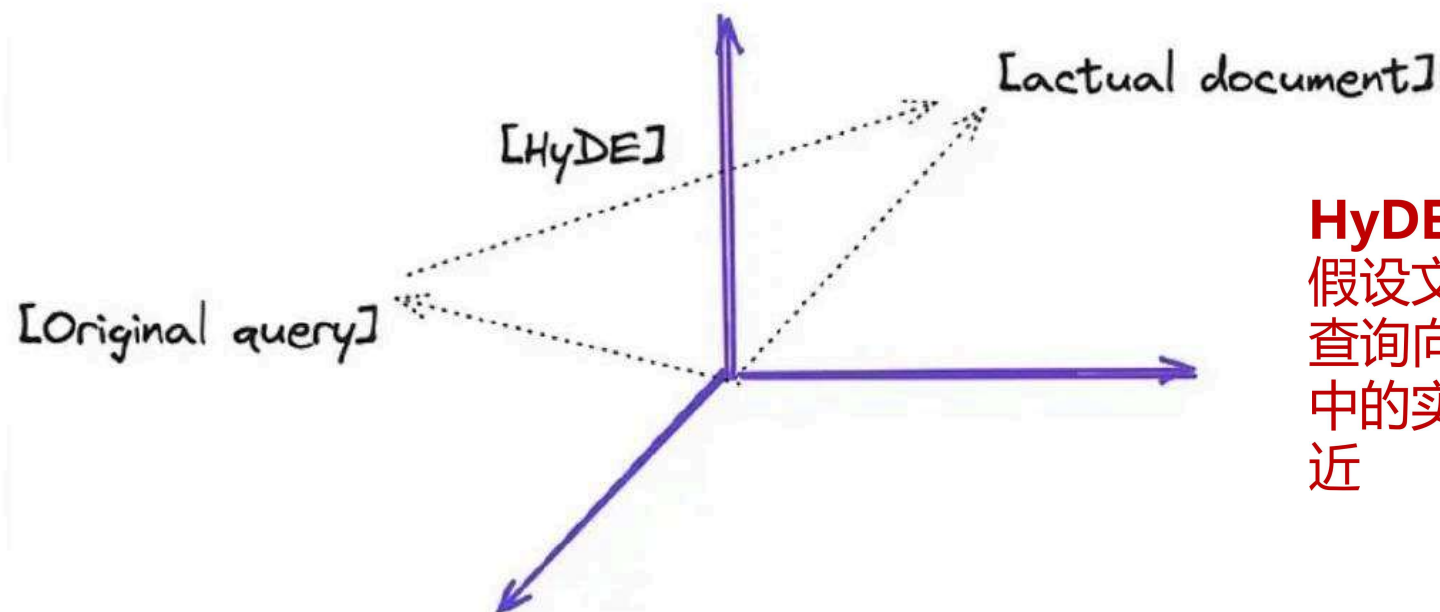
向量化，适合稠密检索

平均化的向量包含了用户查询和所需答案相关的信息，因此能够提高检索的召回率



检索增强技术

◆ 查询增强：HyDE（假设性文档向量）



HyDE 的目标：是生成假设文档，使得最终的查询向量 v 与向量空间中的实际文档尽可能接近

Hypothetical Document Embeddings (HyDE)

检索增强技术

◆ 查询增强：HyDE（假设性文档向量）-Langchain实现

```
prompt_template = """Please answer the user's question about the most recent state of the union address
Question: {question}
Answer: """
prompt = PromptTemplate(input_variables=["question"], template=prompt_template)
llm_chain = LLMChain(llm=llm, prompt=prompt)
```

```
embeddings = HypotheticalDocumentEmbedder(
    llm_chain=llm_chain, base_embeddings=base_embeddings
)
```

```
result = embeddings.embed_query(
    "What did the president say about Ketanji Brown Jackson"
)
```

embedding

检索增强技术

◆ 查询增强：HyDE（假设性文档向量）-llamaindex实现

```
hyde = HyDEQueryTransform(include_original=True)
hyde_query_engine = TransformQueryEngine(query_engine, hyde)
response = hyde_query_engine.query(query_str)
display(Markdown(f"<b>{response}</b>"))
```

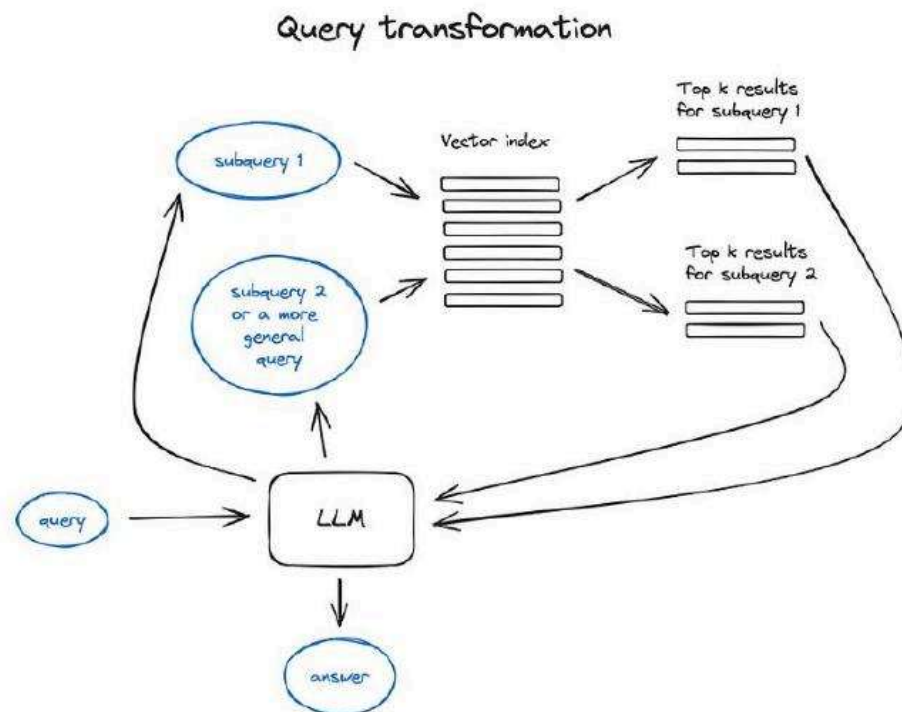
```
"Please write a passage to answer the question\n"
"Try to include as many key details as possible.\n"
"\n"
"\n"
"{context_str}\n"
"\n"
"\n"
'Passage: ""'\n'
```

检索增强技术

◆ 查询增强：子问题查询

问题复杂：问题描述复杂，包含多个语义的问题，直接用复杂问题进行向量化检索可能会导致部分问题丢失，影响检索的性能

问题拆解：利用大语言模型将问题拆解成子问题，每个子问题都生成检索上下文，可以根据合并后的总的上下文去回答，也可以每个上下文独立去回答



检索增强技术

◆ 查询增强：子问题查询-子问题拆解

Your purpose is to help answer a complex user question by generating a list of sub \ questions that can be answered by the tools.

These are the guidelines you consider when completing your task:

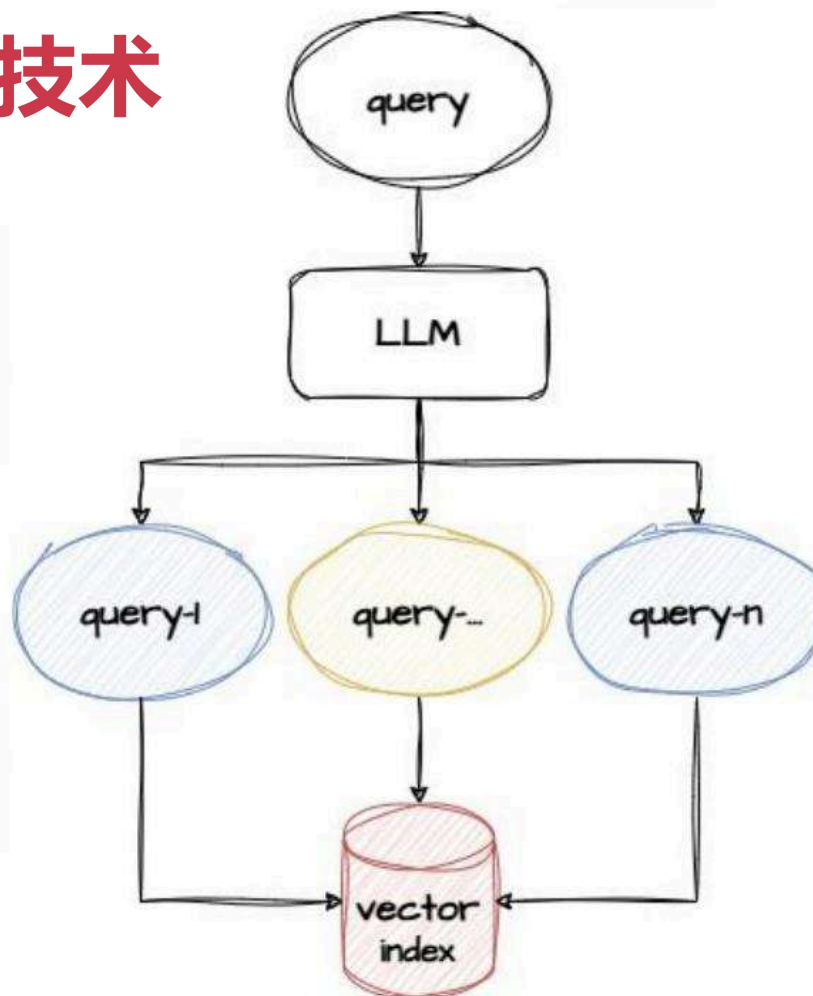
- * Be as specific as possible
- * The sub questions should be relevant to the user question
- * The sub questions should be answerable by the tools provided
- * You can generate multiple sub questions for each tool
- * Tools must be specified by their name, not their description
- * You don't need to use a tool if you don't think it's relevant

检索增强技术

◆ 查询增强：查询改写

问题表达不清：用户提出的问题，语言措辞不好，缺少关键的信息，降低检索性能

问题改写：利用大语言模型的能力，根据用户问题多角度重写问题，增加额外的信息，提高检索性能



检索增强技术

◆ 查询增强：查询改写-实现

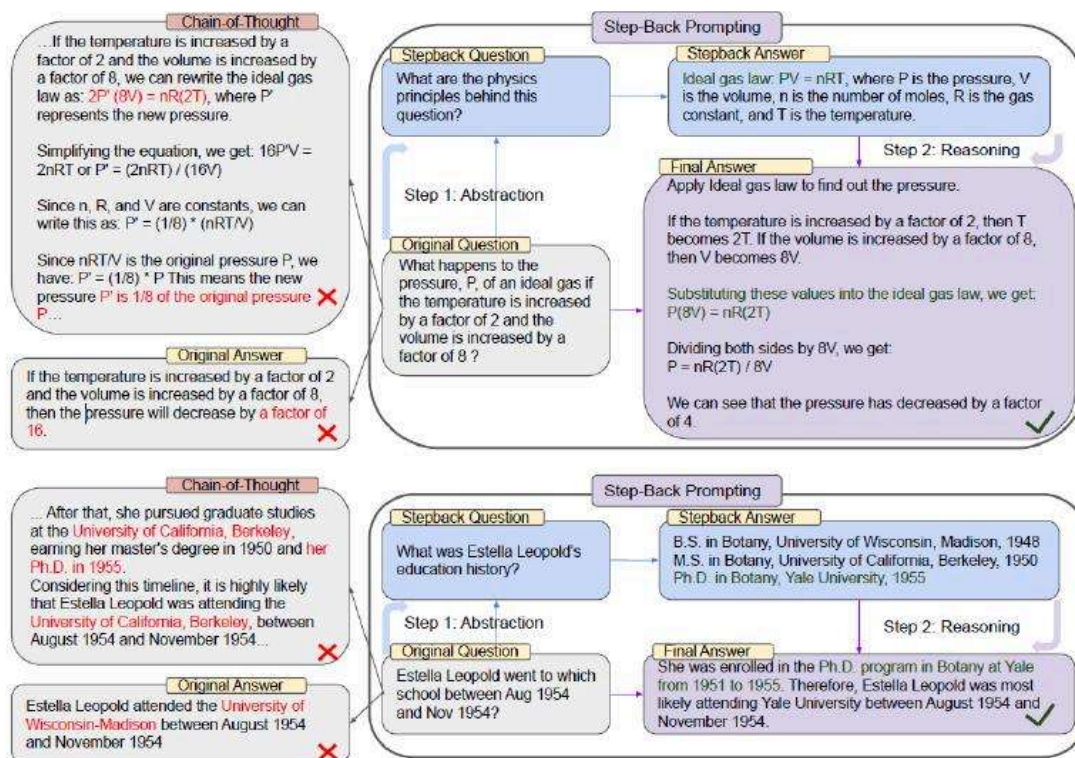
```
QUERY_PROMPT = PromptTemplate(  
    input_variables=["question"],  
    template="""You are an AI language model assistant. Your task is to generate five  
different versions of the given user question to retrieve relevant documents from a vector  
database. By generating multiple perspectives on the user question, your goal is to help  
the user overcome some of the limitations of the distance-based similarity search.  
Provide these alternative questions separated by newlines.  
Original question: {question}""",  
)
```

检索增强技术

◆ 查询增强：Take a Step Back

问题太细：用户提出的问题往往包含大量细节，可能导致检索时忽略了关键的信息，降低检索性能

问题抽象化：将用户的具体问题转化为一个更高层次的抽象问题，一个更广泛的问题，关注于高级概念或原则，从而帮助模型更有效地进行信息检索



检索增强技术

◆ 查询增强：Take a Step Back-提示词

You are an expert at world knowledge.

Your task is to step back and paraphrase a question to a more generic step-back question, which is easier to answer.

Here are a few examples:

Original Question: Which position did Knox Cunningham hold from May 1955 to Apr

Stepback Question: Which positions have Knox Cunningham held in his career?

Original Question: Who was the spouse of Anna Karina from 1968 to 1974?

Stepback Question: Who were the spouses of Anna Karina?

Original Question: Which team did Thierry Audel play for from 2007 to 2008?

Stepback Question: Which teams did Thierry Audel play for in his career?

检索增强技术

◆ 检索增强：多索引检索

多索引检索

用户的查询可能涉及不同的方面和层次，如具体事实、抽象概念等

通过对同一个文档提供不同的类型的embedding索引，能从不同的角度去匹配检索，提高检索匹配的性能

Pre-Retrieval: 检索前增强

检索增强技术

◆ 检索增强：多索引检索-父子文档 (small2big)

文档块分成更小的块small_chunk

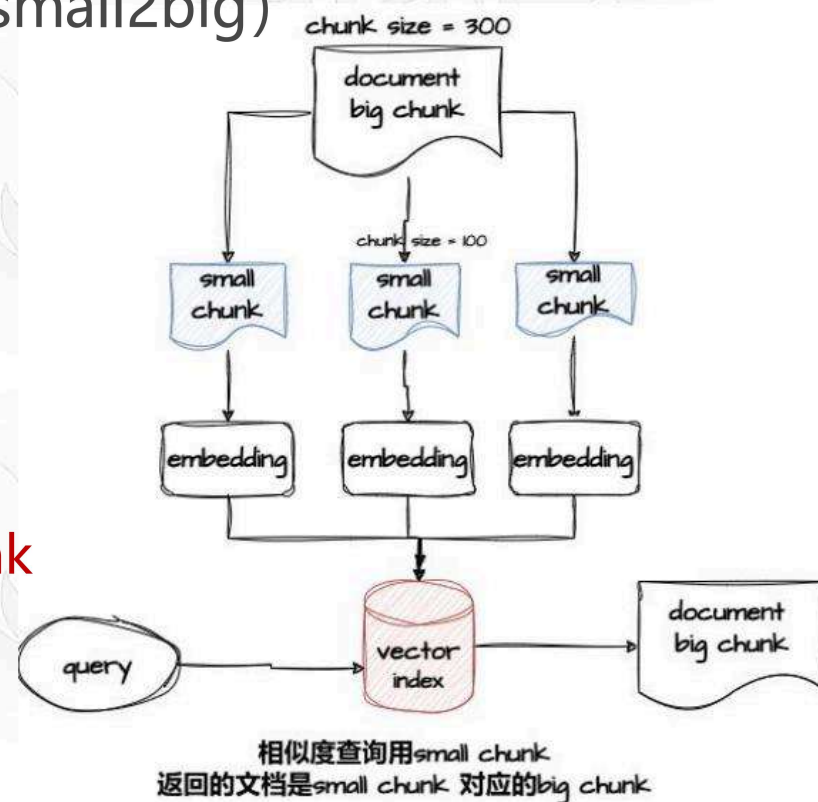


在small_chunk上做embedding



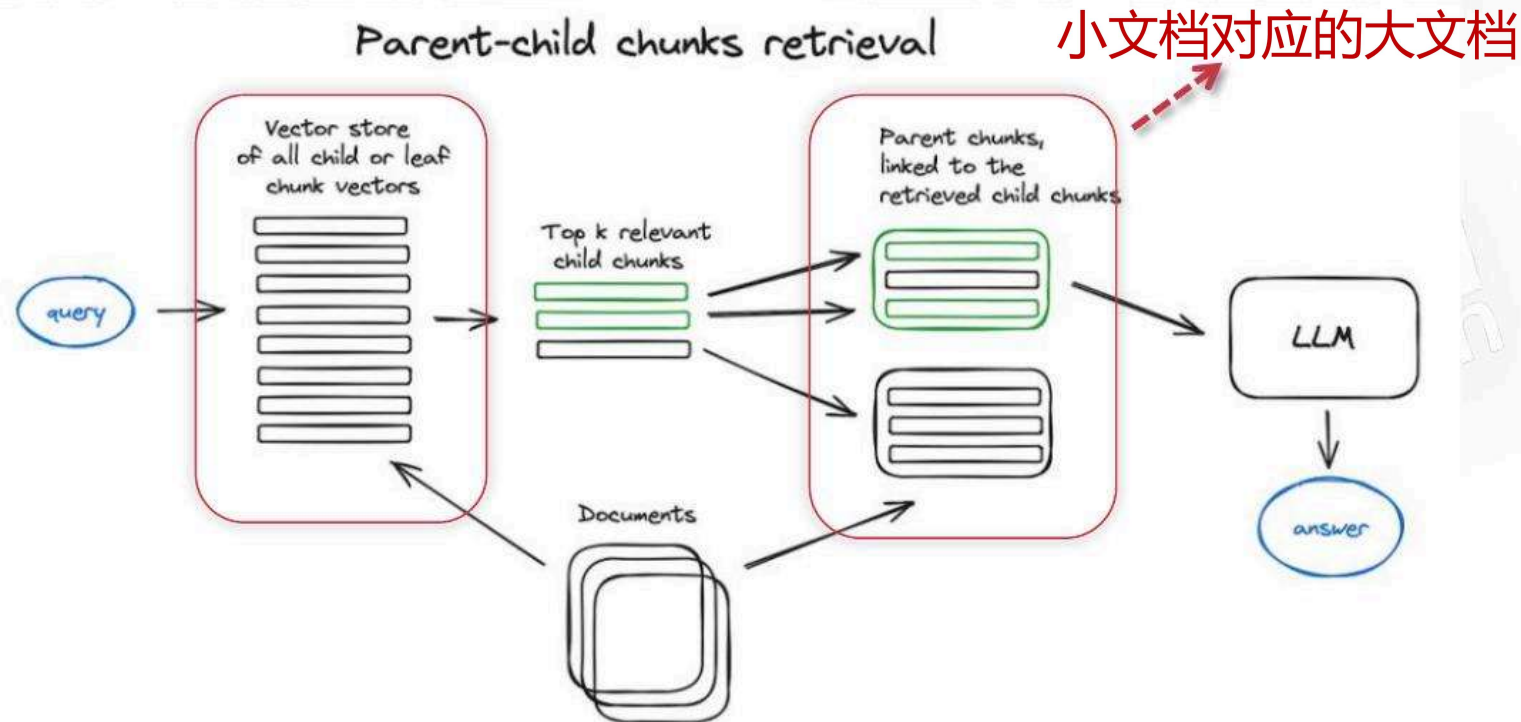
相似度查询用small chunk
返回的文档是small chunk 对应的big chunk

经验表明：开源embedding模型对于短文本提取语义效果更好



检索增强技术

◆ 检索增强：多索引检索-父子文档 (small2big)



检索增强技术

◆ 检索增强：多索引检索-总结索引

文档块生成**总结摘要**

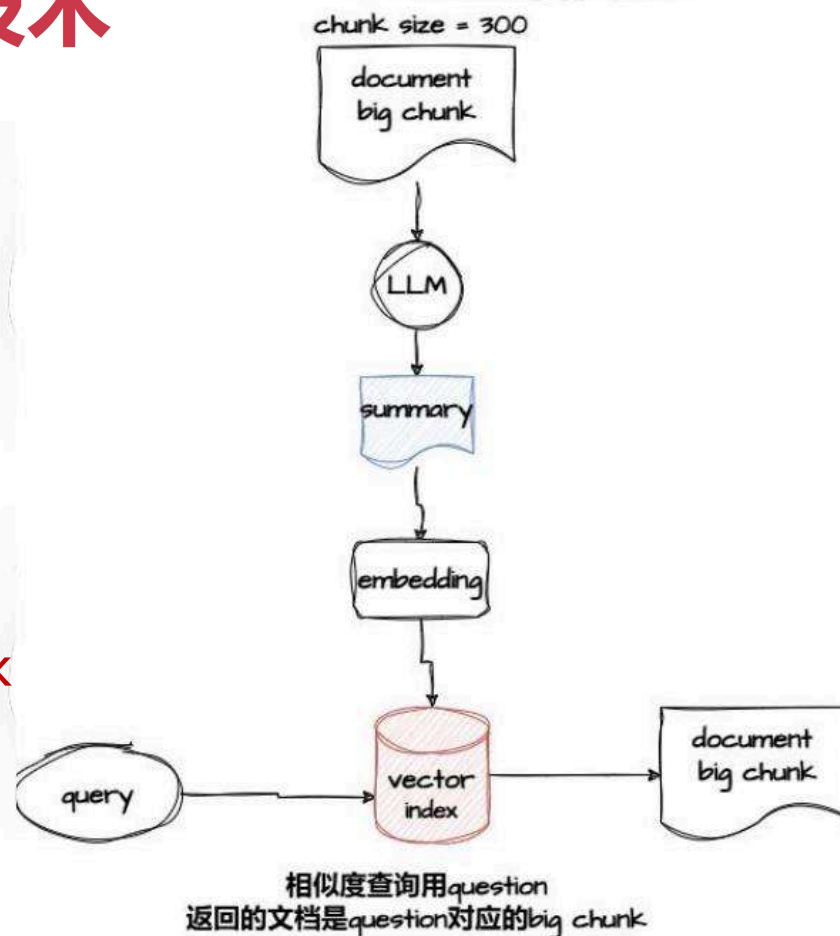


在**总结摘要**上做embedding



相似度查询用**总结摘要**
返回的文档是**总结摘要**对应的big chunk

总结摘要：从信息摘要角度检索文档



检索增强技术

◆ 检索增强：多索引检索-文档假设问题

文档块生成**假设问题**（答案在文档中）

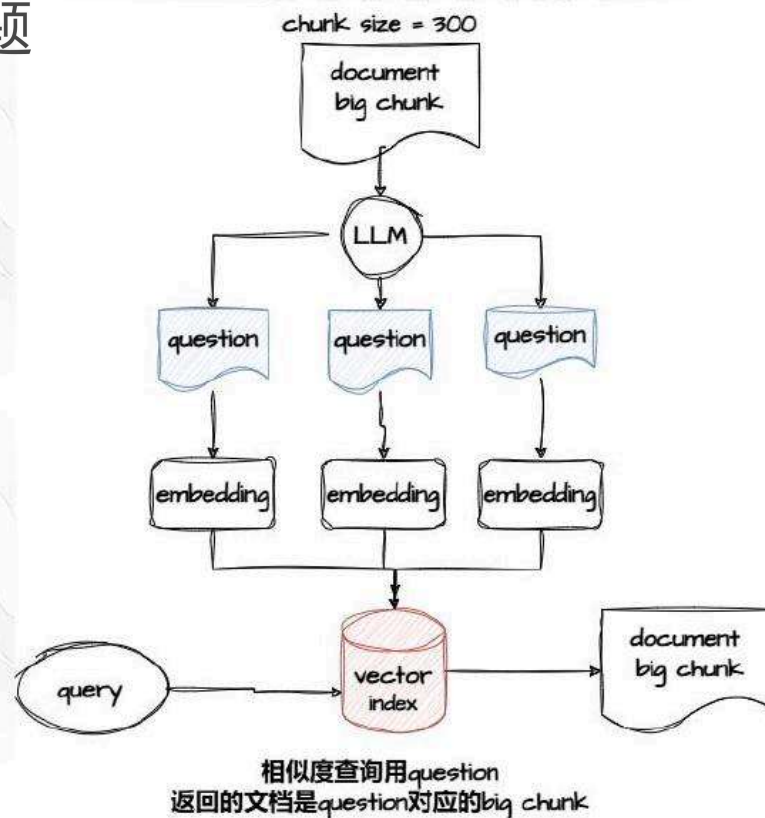


在**假设问题**上做embedding



相似度查询用**假设问题**
返回的文档是**假设问题**对应的big chunk

假设问题：用户问题和假设问题的同一层面的语义匹配



检索增强技术

◆ 检索增强：融合检索（混合检索）

融合检索

基于文本匹配的稀疏检索：关注关键词匹配

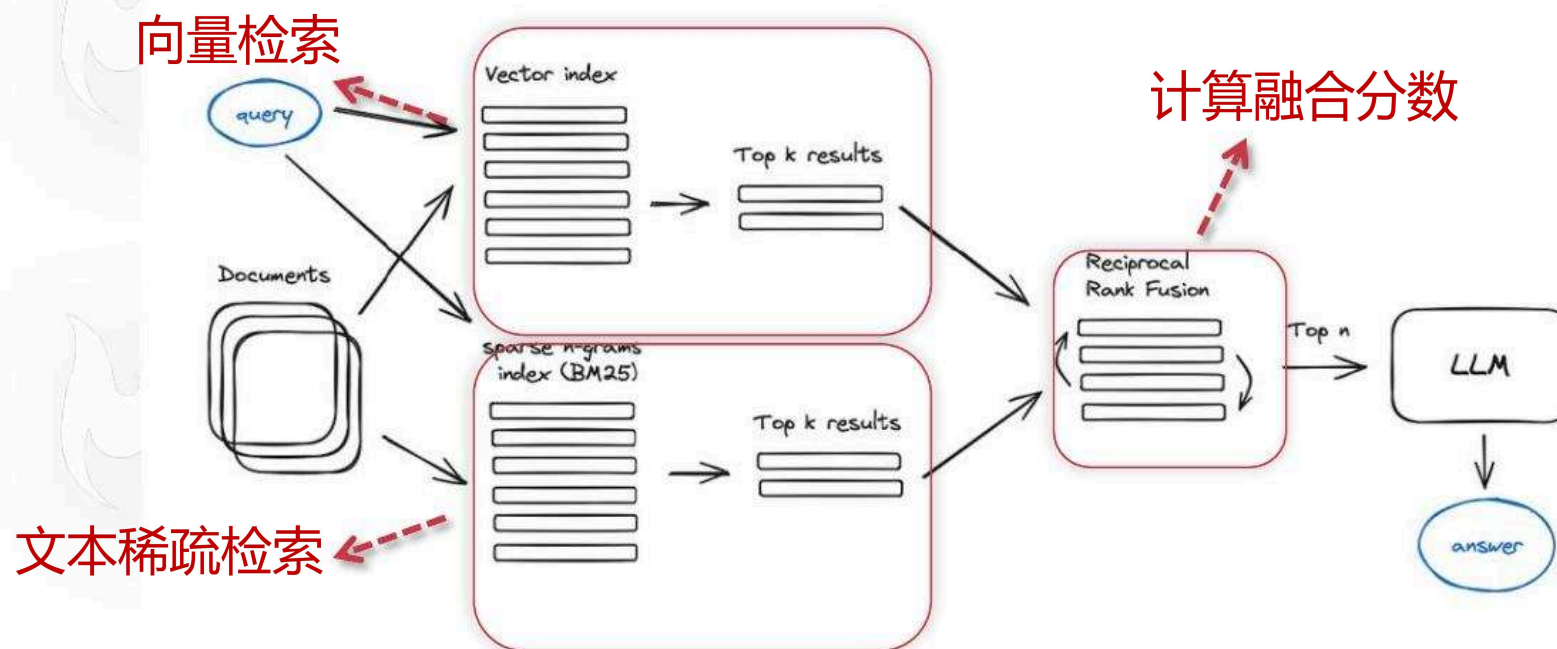
基于语义的稠密检索：关注语义理解

Post-Retrieval: 检索后增强

检索增强技术

◆ 检索增强：融合检索-BM25+embedding

Fusion retrieval / hybrid search



检索增强技术

- ◆ 检索增强：融合检索-融合排序（加权倒数排名）

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)}$$

越相似的文档
权重越高

经验常数:60

单次检索的文档
排序序号
排序越小越相似

检索增强技术

- ◆ 检索增强：融合检索-融合排序（加权倒数排名）

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)},$$

(总体权重0.3)
稀疏检索文档: [A B C] -----> A=1/61 B=1/62 C=1/63]

(总体权重0.7)
稠密检索文档: [C B A] -----> A=1/63 B=1/62 C=1/61]



$$A = 1/61 * 0.3 + 1/63 * 0.7 = 0.160$$

$$B = 1/62 * 0.3 + 1/62 * 0.7 = 0.161$$

$$C = 1/61 * 0.3 + 1/63 * 0.7 = 0.160$$

检索增强技术

◆ 检索增强：重排序rerank

重排序

检索增强：生成多个基于相似度查询到的相似文档

需要精简：检索太多文档增加LLM生成准确答案难度，生成速度慢，引入干扰

文档重排序：使与用户输入问题的更相关、更准确的文档分块排名更靠前

Post-Retrieval: 检索后增强

检索增强技术

◆ 检索增强：重排序rerank

重排序



基于深度学习模型的rerank：一般是Cross-encoder rerank（交叉编码器重排序），它直接对输入的两个文档（即查询问题和候选文档）进行相关性打分，而不是生成句子embedding

常用模型：Cohere Rerank模型 /bge-reranker-large模型

检索增强技术

◆ 检索增强：重排序rerank-bge-reranker

```
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('BAAI/bge-reranker-large')
model = AutoModelForSequenceClassification.from_pretrained('BAAI/bge-reranker-large')
model.eval()

pairs = [['what is panda?', 'hi'],
         ['what is panda?', 'The giant panda (Ailuropoda melanoleuca), sometimes called a panda bear or simply panda, is a bear species endemic to China.']]
with torch.no_grad():
    inputs = tokenizer(pairs, padding=True, truncation=True, return_tensors='pt',
                      max_length=512)
    scores = model(**inputs, return_dict=True).logits.view(-1, ).float()
    print(scores)
```

检索增强技术

◆ 系统性查询增强：迭代检索增强生成

迭代检索增强

单步增强变多次迭代增强，每次迭代都是一次完整的RAG过程，生成答案

每次迭代生成的答案中都可能包含了回答问题的额外信息，这些信息有助于检索更准确的上下文

Modular RAG: 系统性增强

检索增强技术

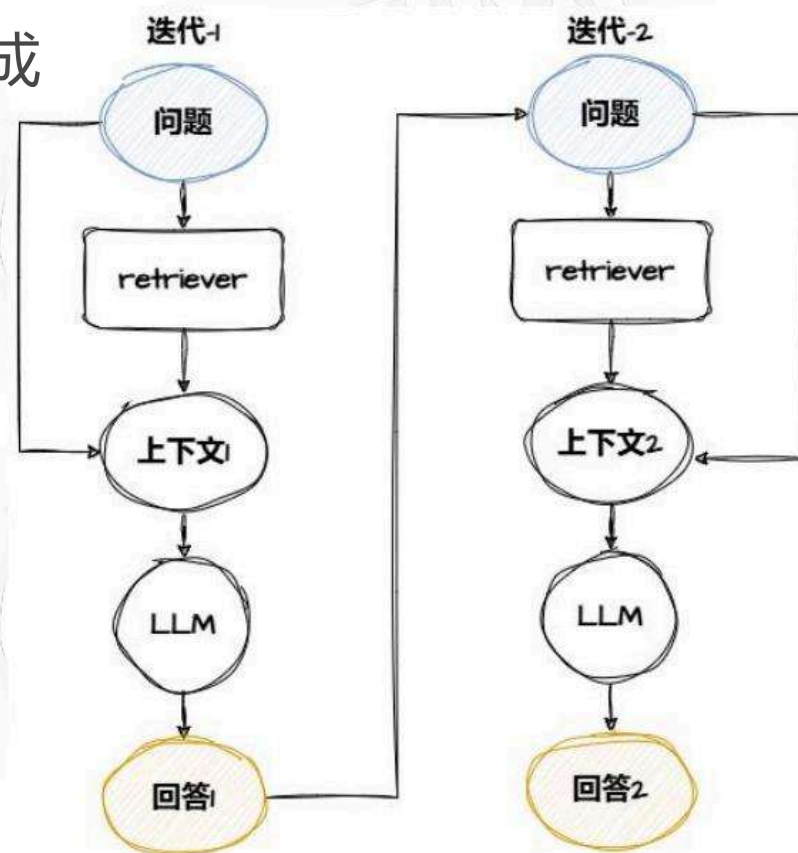
◆ 系统性查询增强：迭代检索增强生成

迭代1： 给定一个问题，检索到上下文信息

迭代1： 模型针对问题和上下文信息，生成可能正确的答案

迭代2： 将生成可能正确的答案再合并到问题，做为新的问题，去检索上下文问题

迭代2： 模型再次针对问题和新的上下文信息，生成答案



检索增强技术

◆ 系统性查询增强：迭代检索增强生成

Question: q

What is the height of the player who won the 2015 AFL Rising Star award?

Retrieval: $q \rightarrow \mathcal{D}_q$

Title: 2015 AFL Rising Star Context: The NAB AFL Rising Star award is given annually to a stand out young player in the Australian Football League. The award was won by Jesse Hogan of Melbourne

Retrieval-Augmented Generation: $\mathcal{D}_q \parallel q \rightarrow y_1$

The 2015 AFL Rising Star award was won by Jesse Hogan of Melbourne. Jesse Hogan is a professional Australian rules footballer. He is 198 cm tall. So the answer is 198 cm

Iteration 1

Question: q

What is the height of the player who won the 2015 AFL Rising Star award?

Generation-Augmented Retrieval: $y_1 \parallel q \rightarrow \mathcal{D}_{y_1 \parallel q}$

Title: Jesse Hogan Context: Jesse Hogan ... playing for the Melbourne Football Club. A key forward, Hogan is 1.95 m tall ... made his AFL debut in the 2015 season and won the Ron Evans Medal as the AFL Rising Star

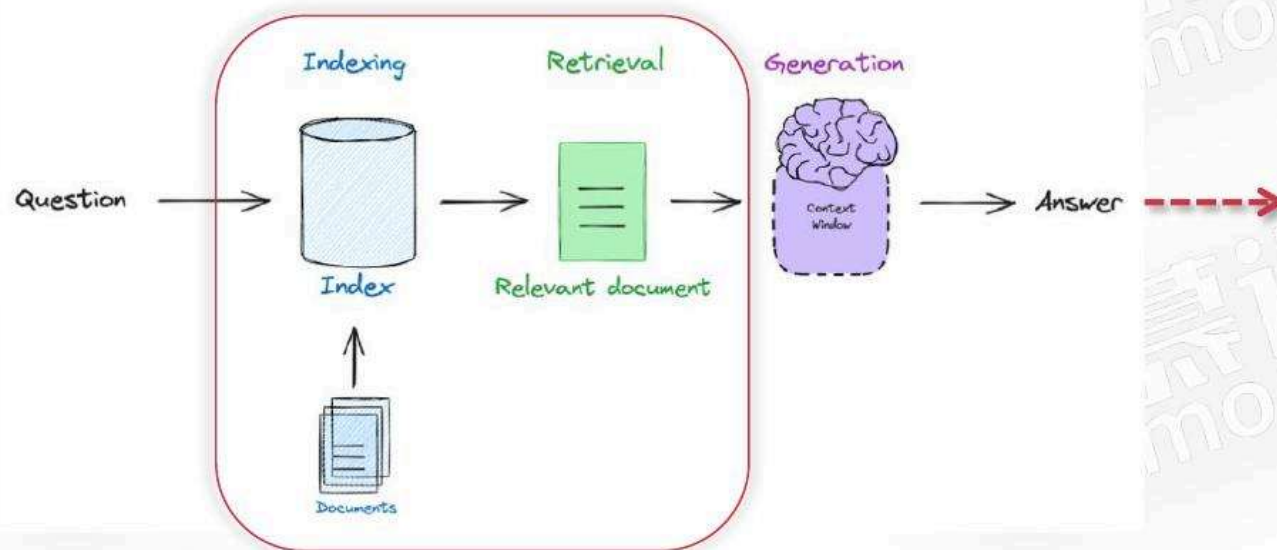
Retrieval-Augmented Generation: $\mathcal{D}_{y_1 \parallel q} \parallel q \rightarrow y_2$

The 2015 AFL Rising Star award was won by Jesse Hogan of Melbourne. Jesse Hogan is 1.95 m tall. So the answer is 1.95 m

Iteration 2

检索增强技术

◆ RAG新范式：self-RAG



查询增强+检索增强

人为/ragas
结果评估

线下迭代不断
优化：确定完
优化步骤，上
线以后这些操
作都是固定

检索增强技术

◆ RAG新范式：self-RAG

问题：RAG流程是否自动加入判别的步骤，可以动态的选择使用哪种策略，自动迭代生成最好的答案

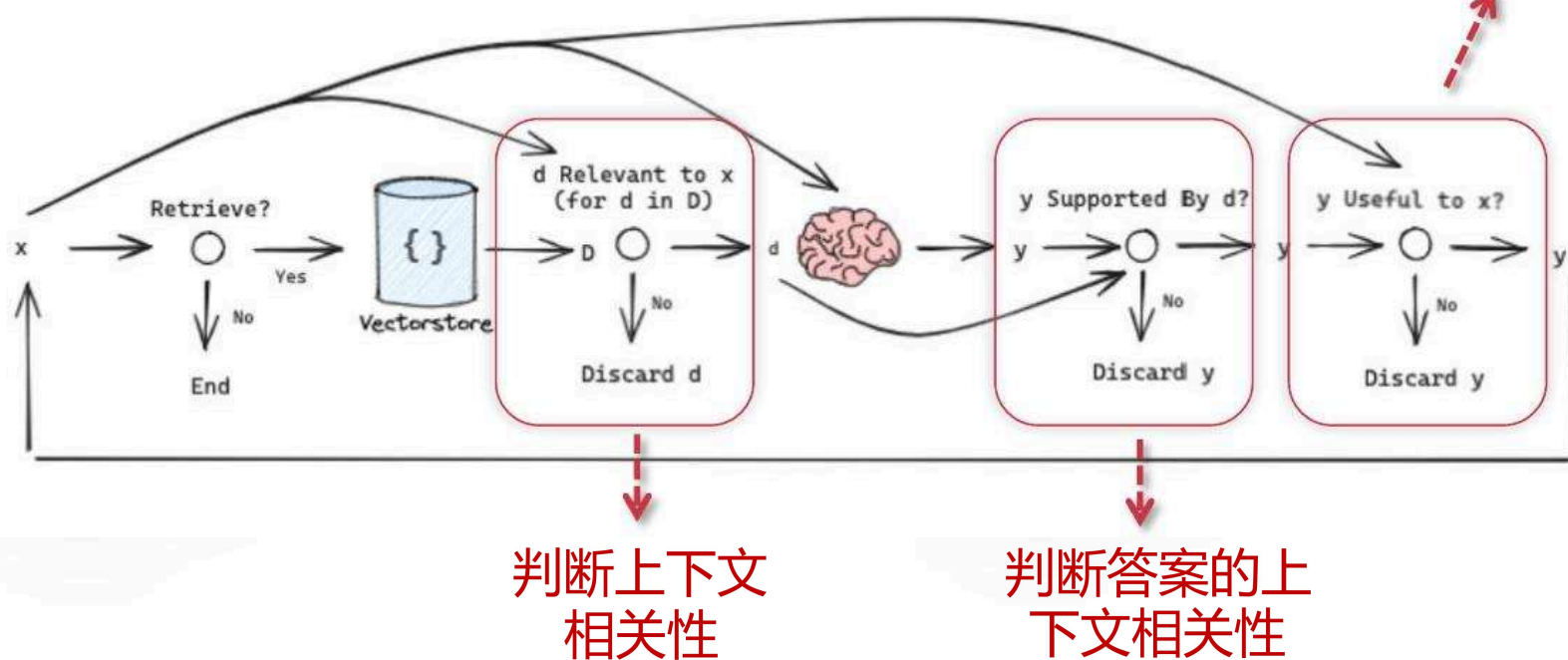
判断检索的文档是否相关：只取相关的，都不相关采用查询增强策略

判断生成答案是否符合上下文：不符合可以多次生成尝试

判断生成答案是否能回答问题：不能回答直接告知或采用新的增强策略

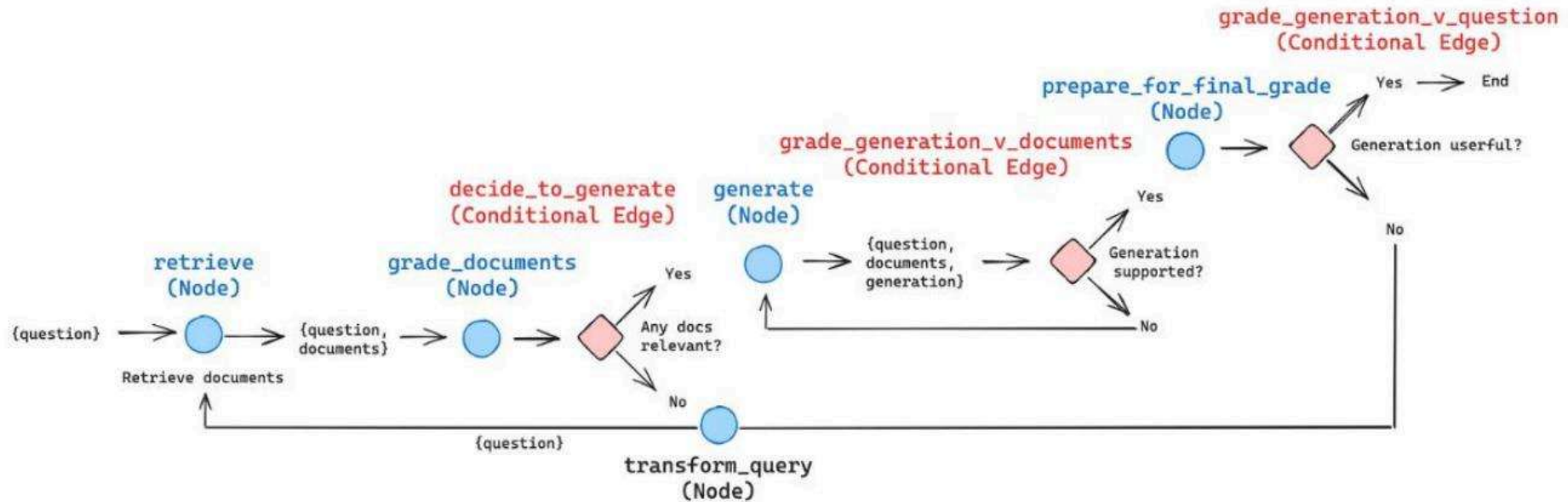
检索增强技术

◆ RAG新范式：self-RAG



检索增强技术

◆ RAG新范式: self-RAG



Langchain: langgraph

总结和展望

◆ 总结

RAG检索增强

RAG设计框架：Navie RAG、Advanced RAG和Modular RAG

检索形态：基于文本匹配统计的稀疏检索（TF-IDF和BM25），基于语义embedding的稠密检索（BGE/GTE）

用户问题的多样性：引出检索增强技术

总结和展望

◆ 总结

查询增强

目的：从用户问题出发对查询进行转换，

Query2Doc, HyDE：查询生成假设性的文档，来增加额外的信息来增加上下文的匹配

查询改写，子问题查询：对问题进行重写，对复杂问题进行拆解

Take step back：对问题进行抽象化处理，去除无畏的细节，增加匹配度

总结和展望

◆ 总结

多索引检索

父子检索：从文档的大小颗粒度来建立索引，用小文档来检索，返回大文档用做上下文

总结检索：从文档的信息摘要来建立索引，用大文档的摘要来检索，返回大文档用做上下文

假设性问题检索：从问题这一个颗粒度来建立索引，实现问题和问题的匹配检索，用大文档逆向衍生出来的不同角度的假设性问题来检索，返回大文档用做上下文

总结和展望

◆ 总结

检索后增强

融合检索：综合基于关键词匹配的稀疏检索和基于语义的稠密检索的特点，重点是对两种检索方法得到的文档，根据加权倒数排名来计算融合分数，以此来确定融合后的文档的排序

重排序rerank：对于检索到众多文档，利用模型的能力来评分，评分越高和问题越相关，**rerank**是在基于相似度检索的基础上，从另外一种维度来对检索文档进行精排

总结和展望

◆ 总结

模块化系统增强

迭代检索增强生成：从迭代的角度利用每次迭代的生成的答案来进行增加检索的相关性，以此来提升检索的性能

Self-RAG：RAG新范式，将评判步骤添加到RAG的流程中，使得可以根据RAG流程每个步骤的结果的好坏来动态的选择优化策略

总结和展望

◆ 关于企业里需要良好的代码规范和代码管理

代码规范

项目开始要做的事项：一般项目成立之初，都会对项目的代码开发做一定的规范约定

规范约定：保持项目的一致性风格，促进项目团队协作和代码维护，提高工作效率

个人的专业形象：形成自己好的代码规范是一个优秀开发者的能力的体现

总结和展望

◆ 关于企业里需要良好的代码规范和代码管理

代码规范约定

命名约定：规定如何为变量、函数、类、文件等命名，驼峰（camelCase）或下划线（snake_case）

代码格式：包括缩进风格（如使用空格或制表符）、每行字符的最大长度、空行的使用等

注释：指导如何编写注释，以便解释复杂的逻辑或算法。注释应当简明扼要，并为其他开发者提供有用的信息

总结和展望

◆ 关于企业里需要良好的代码规范和代码管理

代码规范约定

代码结构：涵盖代码的组织方式，包括文件和目录的布局、模块化设计、类和函数的结构等

错误处理：规定如何进行错误处理和异常捕获，以提高代码的稳健性和可靠性

设计规范原则：开闭原则、单一职责原则、接口隔离原则、迪米特法则、里氏替换原则、依赖倒置原则

总结和展望

Google Python的
开发规范说明

◆ 关于企业里需要良好的代码规范和代码管理

2.11.4 建议

简单情况可以使用.每个部分(真值表达式,if表达式,else表达式)必须在一行内完成.如果使用条件表达式很高的时候使用完整的if语句.

Yes:

```
one_line = 'yes' if predicate(value) else 'no'
slightly_split = ('yes' if predicate(value)
                  else 'no, nein, nyet')
the_longest_ternary_style_that_can_be_done = (
    'yes, true, affirmative, confirmed, correct'
    if predicate(value)
    else 'no, false, negative, nay')
```

No:

```
bad_line_breaking = ('yes' if predicate(value) else
                     'no')portion_too_long = ('yes'
        if some_long_module.some_long_predicate_function(
            really_long_variable_name)
        else 'no, false, negative, nay')
```

<https://github.com/shendeguiize/GooglePythonStyleGuideCN>

总结和展望

Google C++ 的开发规范说明

◆ 关于企业里需要良好的代码规范和代码管理

```
// Copyright 2008 Google Inc.
// License(BSD/GPL/MIT/...)
// Author: voidccc
// This is ...

#include "eventloop.h"
#include <sys/types.h>
#include <vector>
#include "base/basictypes.h"
#include "foo/public/bar.h"

using std::string;

namespace mynamespace {

EventLoop::EventLoop()
: _num_entries(10),
  _num_completed_connections(false) {}
...

Return type ClassName::ReallyLongFunctionName(const Type& par_name1,
                                                Type* par_name2) {
    bool retval = DoSomething(overlyveryveryverylongargument1,
                              argument2, argument3);
    if((condition)) {
        for(int i = 0; i < kSomeNumber; ++i) {
            if (this_one_thing > this_other_thing) {
                a_third_thing -- a_fourth_thing;
                // 1000(name@abc.com): xxx
            }
        }
    } else {
        nt j = 0;
    }
    switch(var) {
        case 0:
            break;
        default:
            assert(false);
    }
    return x;
}
// namespace mynamespace
```

版权
许可证, BSD/GPL/MIT/...
作者
文件内容简短说明

包含及序标准化避免依赖依赖:
1 本类的声明(第一个包含本类头文件,有效减少依赖)
2 C系统文件
3 C++系统文件
4 其他头文件
5 本模块内头文件(避免使用UNIX文件路径"/"和".")

可以在整个cc文件和h文件的方法内使用using
禁止使用using namespace xxx污染命名空间

多行初始化列表, "1"前4空格缩进, 以","结束
多个变量折行对齐
单行初始化列表 Class::Class() : _var(xxx) {}
构造函数中只进行那些没有实际意义的初始化

参数过多时,"结尾
每行一个变量对齐

参数过多时,"结尾
每行一个变量对齐

条件语句内无空格, (condition)左右1空格, if执行保2空格缩进

前置自增运算
条件变量过多时, 条件运算符 && 结尾
条件对齐
暂时方案使用1000(大写)注释,
而前括号里加上你的大名、事件地址等

大括号与else同行, else后右1空格
尽量使用初始化时声明

(var)左右各1空格
条件相对switch 2空格缩进
执行体相对switch 4空格缩进

变量不使用时
不使用静态
禁止使用RTTI
使用printf之类似函数
给值域不使用无符号数字
给特定环境, 不使用操作符重载
使用4种const运算符类型转换
禁止使用Class类安全而变量
若使用必须为类例模式
sizeof(var)代替sizeof(type)
scoped_ptr可以放在智能指针
特殊情况下可用shared_ptr
任何时侯都不使用auto_ptr

返回值不需要加括号

命名空间结束注释

```
// Copyright 2008 Google Inc.
// License(BSD/GPL/MIT/...)
// Author: voidccc
// This is ...

#ifdef PROJECT_EVENTLOOP_H_
#define PROJECT_EVENTLOOP_H_

class Channel;

namespace mynamespace {

class EventLoop {public CallbackInterface {
public:
    // 构造函数
    1 typedefs和enums
    2 常量
    3 构造函数
    4 析构函数
    5 成员函数, 会静态数据成员
    6 成员变量, 会静态成员变量

    typedef vector<int> IntVector;
    enum UriTableErrors {
        ERROR_OUT_OF_MEMORY = 0,
        ERROR_MALFORMED_INPUT,
    };
    explicit EventLoop(const int xx);

    void Add(const std::string& input, Channel* output);

    int num_entries() const { return num_entries_; }
    void set_num_entries(int num_entries) { num_entries_ = num_entries; }

private:
    DISALLOW_COPY_AND_ASSIGN(EventLoop);

    const int kDaysInWeek = 7;
    int num_entries_;
    int num_completed_connections_;

    Channel* channel_;
};
// namespace mynamespace

#endif // PROJECT_EVENTLOOP_H_
```

版权
许可证, BSD/GPL/MIT/...
作者
文件内容简短说明

头文件扩展名.h
文件名全小写, 可包含下划线或短横

防止重复包含 宏格式为: <project>_<path>_<file>_

头文件中尽量使用前置声明
STL类例外不使用前置声明, 使用#include
命名空间全小写 头文件无空格 cc文件提供使用不具命名空间

类名大写开头单词, 使用组合语比使用继承更适宜
若用继承, 只共有继承
另:接口类命名以"Interface"结尾

类内最近按1空格缩进

每一个限定符内, 声明顺序如下:
1 typedefs和enums
2 常量
3 构造函数
4 析构函数
5 成员函数, 会静态数据成员
6 成员变量, 会静态成员变量

2空格缩进
枚举名同类名, 大写开头单词
枚举值2空格缩进, 全大写下划线

explicit修饰非静态构造函数, 防止隐式类型转换
若定义了成员变量无其他构造函数, 定义一个默认构造函数
普通函数命名, 大写开头单词, 输入参数在引号const引用, 输出参数在引号指针
不为参数设置缺省值

普通函数命名, 大写开头单词, 输入参数在引号const引用, 输出参数在引号指针
不为参数设置缺省值

存取函数命名, 取: 同变量名, 存: 值函数名为set_varname
短小的存取函数可用内联

存取函数命名, 取: 同变量名, 存: 值函数名为set_varname
短小的存取函数可用内联

可在类成员函数对象时使用成员构造函数
不需要成员函数在private中使用DISALLOW_COPY_AND_ASSIGN
变量用短缩名称, 不要使用空间, 让别人理解你的代码更重要
const 变量为k开头, 后跟大写开头单词
变量命名: 全小写, 有意义的前缀和下划线
类成员变量下划线结尾

头文件中只用了指针/引用,
则就向声明而非引入头文件

上下少空行, 每屏代码越多越好

左右小于80

保护宏结束加注释

Google C++ Style Guide

总结和展望

◆ 关于企业里需要良好的代码规范和代码管理

代码管理

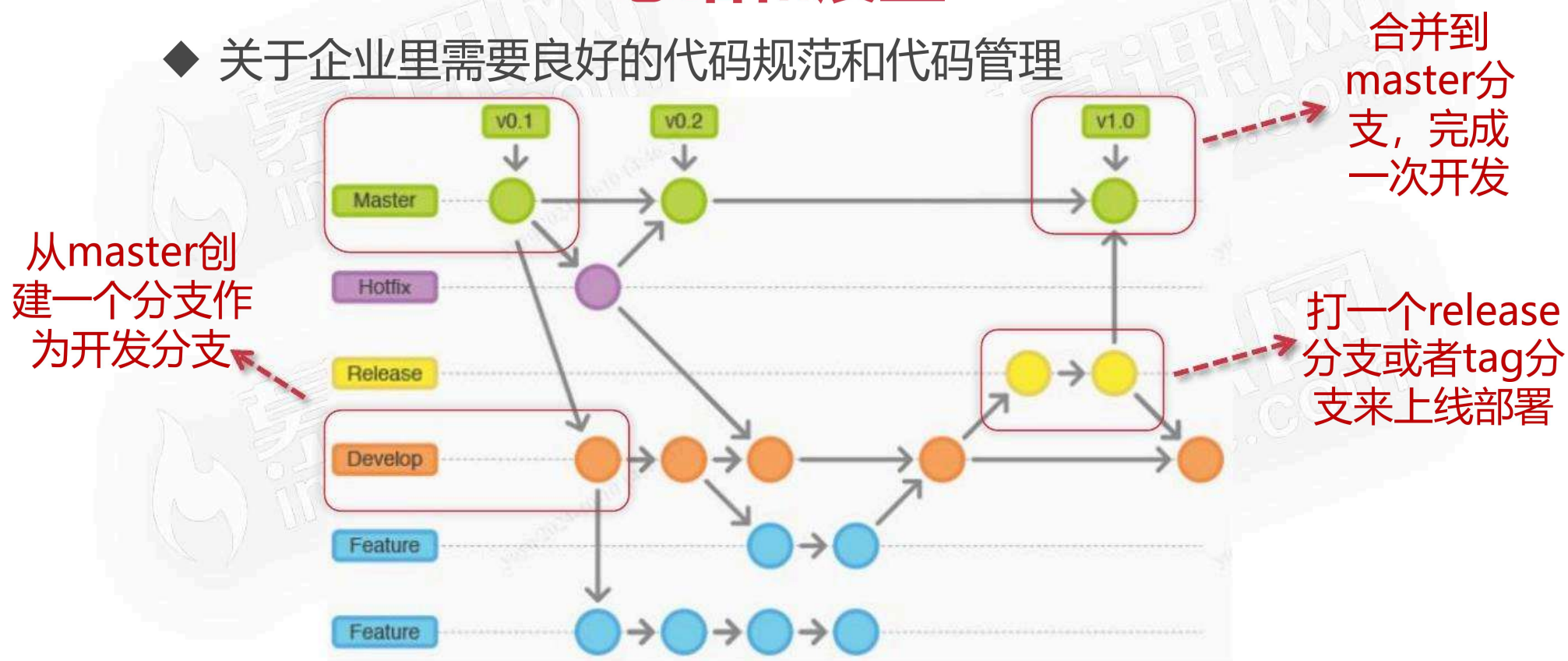
目的：保存代码开发过程的每次变化（commit），方便组织、跟踪和协作开发

代码管理工具：svn/**git**(分布式版本控制系统)

GIT使用：约定分支策略，来协同开发和上线。包括使用master作为生产分支，develop作为开发分支，feature特性分支用于新功能开发，hotfix修复分支用于修复问题

总结和展望

◆ 关于企业里需要良好的代码规范和代码管理



总结和展望

◆ 关于企业里需要良好的代码规范和代码管理

AI应用开发

变化和不确定性：通过迭代去尝试不同的方案，整个过程中需要跟踪尝试了哪些方案，用了哪个提示词

用GIT来管理过程：不同的方案有不同的参数和提示词，这些过程的数据也需要记录下来以便比较，这些操作也可以作为一个小的版本，在git本地仓库中进行管理，不同的方案通过分支名称区分